# Lab Report Task 6

เป้าหมายหลักของ Task นี้คือเพื่อทำความเข้าใจกระบวนการทำงานของลักษณะพื้นฐานของกุญแจสาธารณะ (Public Key Infrastructure - PKI) และ สายใยแห่งความน่าเชื่อถือ (Chain of Trust) ผ่านการตรวจสอบความถูกต้องของใบรับรองอิเล็กทรอนิกส์ชนิด X.509 Certificate ด้วยการตรวจสอบตนเอง

## 1. คำอธิบาย

ในโลกอินเทอร์เน็ตจริง เมื่อเราเข้าเว็บไซต์ เช่น openssl.org ที่ใช้ในใบรับรอง c0.pem ของการทดลองนี้ เบราว์เซอร์ต้องตรวจสอบว่าเว็บไซต์นั้นเป็นตัวจริงหรือไม่ โดยดูจากใบรับรองที่ออกให้โดยหน่วยงานผู้ออกใบรับรอง (CA) ซึ่งในกรณีนี้คือ Google Trust Services (WR3) ที่ดึงใส่ในไฟล์ c1.pem

โดยใน Task นี้ เราจะจำลองตัวเองเป็นเบราว์เซอร์ โดยการทำดึงค่า Public Key ($n$, $e$) ของ CA จากใบรับรอง c1.pem จากนั้นดึงค่า Digital Signature ($S$) ที่ CA เซ็นรับรองไว้ในใบรับรอง c0.pem แล้วเขียนโปรแกรม RSA ถอดรหัสลายเซ็นนั้นตามสูตร $Hash' \equiv S^e (mod\ n)$ จากนั้นนำเนื้อหาของใบรับรองมาคำนวณค่า Hash แบบ SHA-256 ด้วยตนเอง แล้วเปรียบเทียบว่า Hash ที่คำนวณเองตรงกับ Hash ที่ถูกถอดรหัสออกมาจากลายเซ็นของ CA หรือไม่ หากตรงกัน แสดงว่าใบรับรองนี้เป็นของจริงและไม่ถูกดัดแปลง

## 2. ขั้นตอนการสกัดข้อมูล

a. **รันคำสั่งเพื่อ Extract CA Modulus ($n$):** openssl x509 -in c1.pem -noout -modulus

b. **รันคำสั่งเพื่อ Extract CA Exponent ($e$):** openssl x509 -in c1.pem -text -noout (ได้ค่า 65537 หรือ 0x010001)

c. **รันคำสั่งเพื่อ Extract Signature ($S$) from Server Cert:** สกัดจาก openssl x509 -in c0.pem -text -noout และลบเครื่องหมาย : ออกด้วยคำสั่ง tr

d. **รันคำสั่งเพื่อ Extract Certificate Body & Calculate Hash:** openssl asn1parse -i -in c0.pem -strparse 4 -out c0_body.bin -noout

e. **รันคำสั่งเอา hash sha-256:** sha256sum c0_body.bin ได้ผลลัพธ์ Hash c0cf40cca46f93f25cebd141b26758a57b865d15e7f892b0c5c6240fbd4b1013

3. C Source Code

```
#include <stdio.h>
#include <openssl/bn.h>

// Function to print a BIGNUM variable in a readable hexadecimal format
void printBN(char *msg, BIGNUM * a) {
    char * number_str = BN_bn2hex(a); // Convert BIGNUM to hex string
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str); // Prevent memory leak
}
int main () {
    BN_CTX *ctx = BN_CTX_new(); // Allocate context for BIGNUM operations
    BIGNUM *n_ca = BN_new(); // Modulus of the intermediate CA (Google Trust
Services WR3)
    BIGNUM *e_ca = BN_new(); // Public Exponent of the CA
    BIGNUM *s = BN_new();    // Signature extracted from the Server Certificate
(openssl.org)
    BIGNUM *res = BN_new();  // Resulting decrypted signature (Expected to
contain the hash)

    // 1. Assign the public modulus (n) of the CA extracted from c1.pem
    BN_hex2bn(&n_ca,
"8F347587AF8472148D0710916F03ACF1D408359A19F29B1889346C988F7AD4DDEA05E8DE1B7C8C54
12BA798AFB180D0D7C9CF3BD38E4A85EC633CB46896F46A0E937638DDCCCD5974E32AD7B1D2305B9F
57B494398D0BC57C7537818B1EDA754B27C86BEF05445BC87BA99591DF4B8DB00FB814F462B625EB1
3AA52A1723ACA2BEC58EE55EFD711E7DA4B4237D0452B234D2DF99AC87C64C595FF8E64F8E7592C2B
2304692D0B60DC7E48967FF3F54942765E301C84A2C842F655FCDAD5CFDA6AD415BDC4C3F1796917D
A9D83C532A1CD0E6D477E6434AC2B7F848A2CDAD63B5256B96721D81456F8669C4E4E6784C31E6A17
FA701730A87EF878972CCD3C58D");

    // 2. Assign the public exponent (e) of the CA (Standard value: 65537)
    BN_hex2bn(&e_ca, "010001");

    // 3. Assign the digital signature (S) extracted from the Server Certificate
c0.pem
    BN_hex2bn(&s,
"7eaaf7a67112d476f6dbecb83c7d01bc200627ac05e74d0582e3afc5335e48a861ca3da6ebafd2fa
61193d7164fc4a472dd68b0fe915485ef1a2f5d03cc7f14a3db3bfd82e64f66b1b9ad2905fbb87730
0a6fad07cf3e5d4194b769ce9cbd75831b056420b89686232ec68230cc963b2725bcca84262e3b7a8
39bc3fdfd7bd2ec0c5aaab24c064fe3921b334fce8d864288681e67714cc6427220e0a99a77a86531
3b3e666aeeb3ab6e261baf5b0ac51adfed2902ac9e244071da07b8047100fe1fca1910db34ece279f
a7d141c3fd8ccc4df1b578291226bd193efb553f6083992f17cc4c96f60717ea49265c5181576743f
a9dd39a9d80cb0967dc9929e1cf");

    // 4. Perform RSA Verification: res = (S ^ e_ca) mod n_ca
    // This decrypts the CA's signature to reveal the original hash and padding
    BN_mod_exp(res, s, e_ca, n_ca, ctx);

    // Output the decrypted signature
    printf("--- Task 6: Manual X.509 Verification ---\n");
    printBN("Decrypted Signature (Expected Hash inside): ", res);

    // Memory cleanup
    BN_free(n_ca); BN_free(e_ca); BN_free(s); BN_free(res);
    BN_CTX_free(ctx);
    return 0;
}
```

## 4. ผลลัพธ์การรัน

a. ผลการคำนวณ sha256sum c0_body.bin ด้วยตนเองได้ค่า

```
C0CF40CCA46F93F25CEBD141B26758A57B865D15E7F892B0C5C6240FBD4B1013
```

b. ผลลัพธ์จากการรันโปรแกรมภาษา C (Decrypted Signature) ได้ค่า

```
--- Task 6: Manual X.509 Verification ---
Decrypted Signature (Expected Hash inside):
01FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF003031300D0609608648016503
0402010500420C0CF40CCA46F93F25CEBD141B26758A57B865D15E7F892B0C5C6240FBD4B
1013
```

จากผลลัพธ์ข้างต้น จะเห็นได้ค่าคำนวณ sha256 จากการเข้ารหัสตรงกับการ decrypted signature หลังจาก ...420**C0CF..** แสดงว่าสามารถถอดรหัส ส่วนอื่น ๆ จะกล่าวในส่วนถัดไป

## 5. Discussion & Observation (การอภิปรายผล)

จากการตรวจสอบการทำงานและผลลัพธ์ที่ได้ มีข้อสังเกตและการวิเคราะห์เชิงลึกได้ดังนี้

### a. The Verification Success

เมื่อพิจารณาที่ 64 ตัวอักษรสุดท้าย (32 Bytes) ของ Decrypted Signature ที่ได้จากโปรแกรม จะพบค่า *C0CF40CCA46F93F25CEBD141B26758A57B865D15E7F892B0C5C6240FBD4B1013* ซึ่ง**ตรงกัน**กับผลลัพธ์ของการทำ sha256sum จากไฟล์ c0_body.bin ซึ่งมันพิสูจน์ให้เห็นว่าใบรับรองของ openssl.org ออกโดย Google Trust Services จริง และข้อมูลไม่ได้ถูกแอบแก้ไขระหว่างทาง

### b. โครงสร้างของ Decrypted Signature (PKCS#1 v1.5 Padding)

สิ่งที่น่าสนใจอย่างยิ่งคือ ผลลัพธ์จากการถอดรหัสไม่ได้มีแค่ค่า Hash โดดๆ แต่เป็นสตริงยาวๆ สาเหตุเพราะการเข้ารหัส RSA ในมาตรฐาน X.509 ใช้รูปแบบการทำ Padding ที่เรียกว่า **PKCS#1 v1.5** เพื่อให้บล็อกข้อมูลมีขนาดเท่ากับความยาวของกุญแจ RSA (ในที่นี้คือ 2048 บิต) โดยโครงสร้างสามารถแบ่งได้ดังนี้

- **01** บ่งบอกว่าเป็น Block Type 1 ซึ่งสงวนไว้สำหรับการทำ Digital Signature (Private Key Operation)
- **FF FF ... FF** เป็นข้อมูล Padding เพื่อขยายขนาดบล็อกให้เต็ม
- **00** เป็น Byte ตัวคั่น (Separator) บ่งบอกว่าจบช่วง Padding แล้ว
- **3031300D060960864801650304020105000420 (ASN.1 Magic Number)** เป็นค่าพิเศษที่เรียกว่า DigestInfo ที่ถูกเข้ารหัสในรูปแบบ DER/ASN.1 โดยค่ายาว ๆ เหล่านี้ทำหน้าที่ระบุว่า **อัลกอริทึมที่ใช้ทำ Hash คือ SHA-256** โดยหากใบรับรองใช้ SHA-1 หรือ MD5 ค่าตรงนี้จะเปลี่ยนไป
- **C0CF40CC...** เป็นค่า Message Digest (Hash) ตัวจริงจำนวน 32 Bytes

การสังเกตเห็นโครงสร้าง PKCS#1 v1.5 นี้ ทำให้เข้าใจลึกซึ้งขึ้นว่า ในกระบวนการทำงานจริง ซอฟต์แวร์ เช่น เบราว์เซอร์ ไม่ได้เช็คแค่ค่า Hash เพียงอย่างเดียว แต่ต้องตรวจสอบโครงสร้าง Padding และ Algorithm ID ASN.1 ว่าถูกต้องสมบูรณ์ด้วย เพื่อป้องกันการโจมตีทางวิทยาการเข้ารหัสลับประเภทต่าง ๆ ด้วย